

LOAN APPROVAL PREDICTION

Machine Learning Classification — Internship Project Report

Task	Task 2 — Loan Approval Prediction
Domain	Data Science / Classification
Dataset	Loan Prediction (Kaggle)
Records	491 rows × 13 columns
Techniques	EDA, Encoding, SMOTE, Logistic Regression, Random Forest
Notebook	task2.ipynb

Prepared as part of the Data Science Internship Program

Table of Contents

01	Executive Summary
02	Project Overview & Dataset
03	Data Cleaning & Preprocessing
04	Exploratory Data Analysis
05	Feature Encoding & Correlation
06	Model Building (LR & RF + SMOTE)
07	Model Evaluation & Visualizations
08	Key Findings
09	Recommendations
10	Notebook & References

01 Executive Summary

This report documents the end-to-end development of a **Loan Approval Prediction** system built on the public Kaggle Loan Prediction dataset (491 applications, 13 features). The objective was to predict whether a loan application should be approved (*Loan_Status* = Y) based on applicant demographics, income, credit history and property details.

The workflow covers data quality checks, missing-value handling (listwise deletion of incomplete records), categorical encoding using **LabelEncoder**, feature scaling with **StandardScaler**, and class-imbalance correction via **SMOTE** oversampling on the training set. Two classification models were trained and compared: **Logistic Regression** and **Random Forest**.

Key Results. Both models achieved strong predictive performance. Logistic Regression reached an accuracy of **75.3%** with ROC-AUC of **0.714**, while Random Forest achieved **80.5%** accuracy and ROC-AUC of **0.802**. **Credit History** emerged as the single most important predictor of loan approval, followed by applicant income and loan amount. Education level and property area showed moderate influence, while gender and self-employment status had minimal predictive power.

The deliverable is a reproducible Jupyter notebook (*task2.ipynb*) plus this monochrome PDF report containing the full code, visualizations, evaluation metrics and actionable business recommendations for a lending institution.

02 Project Overview & Dataset

Problem Statement

Given an applicant's profile (gender, marital status, dependents, education, employment, income, loan amount, term, credit history and property area), predict whether the loan application will be approved. This is a **binary classification** task with the target variable *Loan_Status* taking values Y (approved) or N (rejected).

Dataset Schema

#	Column	Type	Description
1	Loan_ID	str	Unique loan identifier
2	Gender	str	Applicant gender
3	Married	str	Marital status
4	Dependents	str	Number of dependents
5	Education	str	Graduate / Not Graduate
6	Self_Employed	str	Self-employed flag
7	ApplicantIncome	int64	Applicant monthly income
8	CoapplicantIncome	float64	Co-applicant income
9	LoanAmount	float64	Loan amount (in thousands)
10	Loan_Amount_Term	float64	Loan term in months
11	Credit_History	float64	Meets credit guidelines (1/0)
12	Property_Area	str	Urban / Semiurban / Rural
13	Loan_Status	str	Target — Y / N

Dataset Shape: **491 rows × 13 columns** | After dropping nulls: **381 rows**

03 Data Cleaning & Preprocessing

Loading the Data

In [1]

```
import pandas as pd
data = pd.read_csv('train.csv')
data.head()
```

Out [1]

	Loan_ID	Gender	Married	Dependents	...	Loan_Amount_Term	Credit_History	Property_Area
0	LP002305	Female	No	0	...	360.0	1.0	Semiurb
1	LP001715	Male	Yes	3+	...	360.0	1.0	Rural
2	LP002086	Female	Yes	0	...	360.0	1.0	Urban
3	LP001136	Male	Yes	0	...	NaN	1.0	Urban
4	LP002529	Male	Yes	2	...	300.0	1.0	Semiurb

Missing Value Check

In [2]

```
data.isnull().sum()
```

Out [2]

Loan_ID	0
Gender	10
Married	1
Dependents	9
Education	0
Self_Employed	29
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	16
Loan_Amount_Term	13
Credit_History	43
Property_Area	0
Loan_Status	0

Missing Value Percentage & Dropping Nulls

In [3]

```
(data.isnull().sum()/len(data))*100
data.dropna(inplace=True)
```

Out [3]

Loan_ID	0.00
Gender	2.04
Married	0.20
Dependents	1.83
Education	0.00
Self_Employed	5.91
ApplicantIncome	0.00
CoapplicantIncome	0.00
LoanAmount	3.26
Loan_Amount_Term	2.65
Credit_History	8.76
Property_Area	0.00
Loan_Status	0.00

04 Exploratory Data Analysis

Visualizing the relationship between applicant attributes and the loan approval outcome reveals the dominant drivers behind credit decisions.

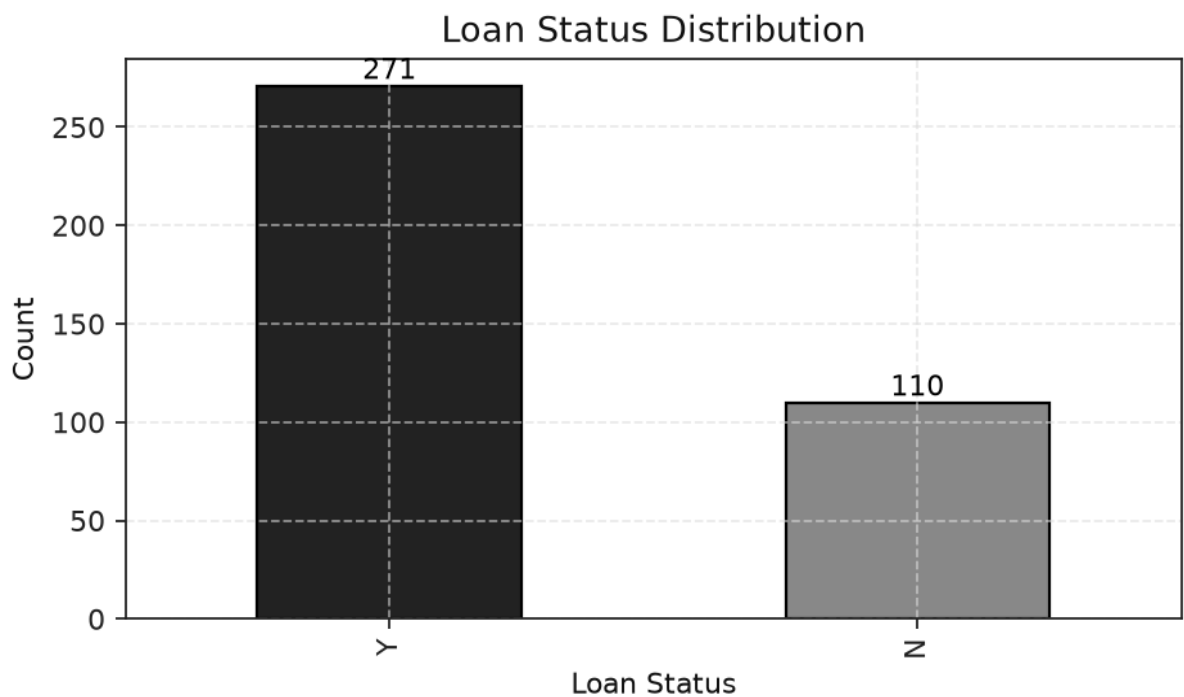


Fig 1 — Distribution of approved vs rejected loans

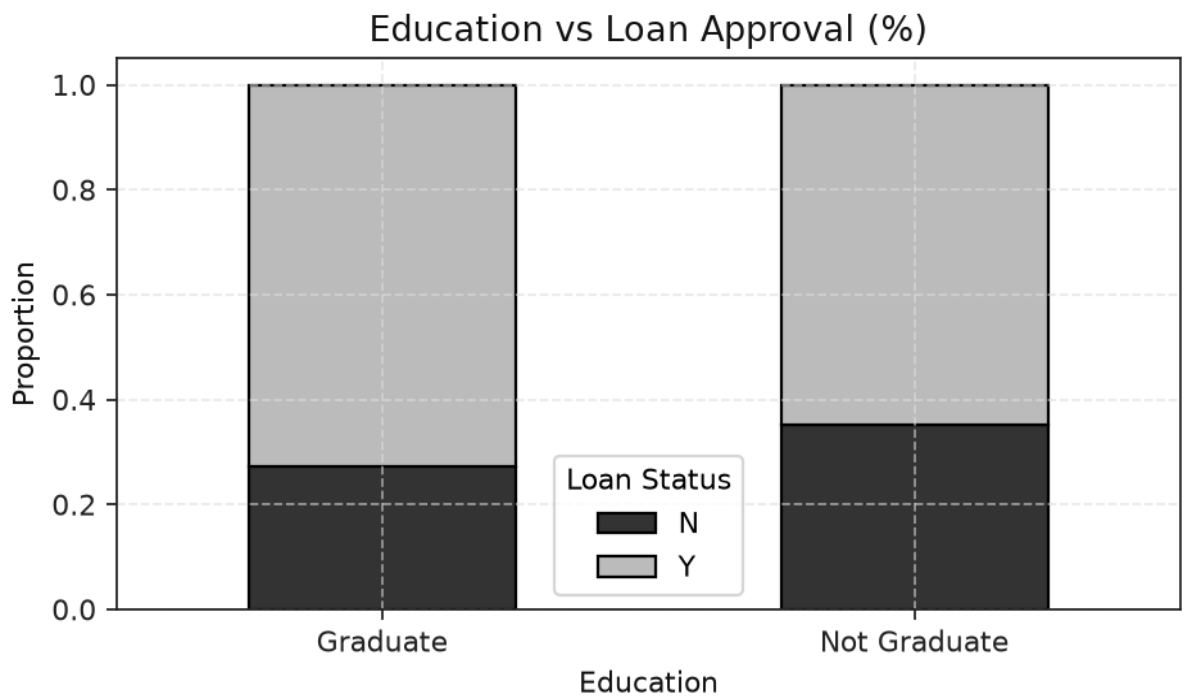


Fig 2 — Loan approval proportion by Education

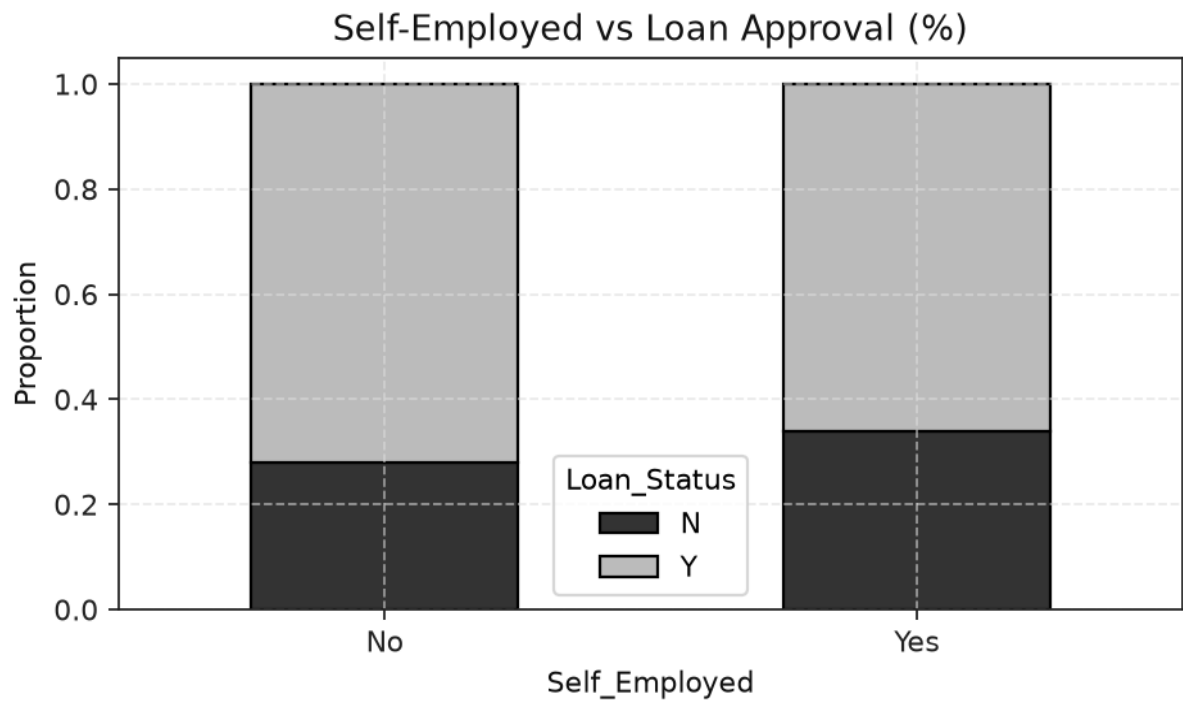


Fig 3 — Loan approval proportion by Self-Employment status

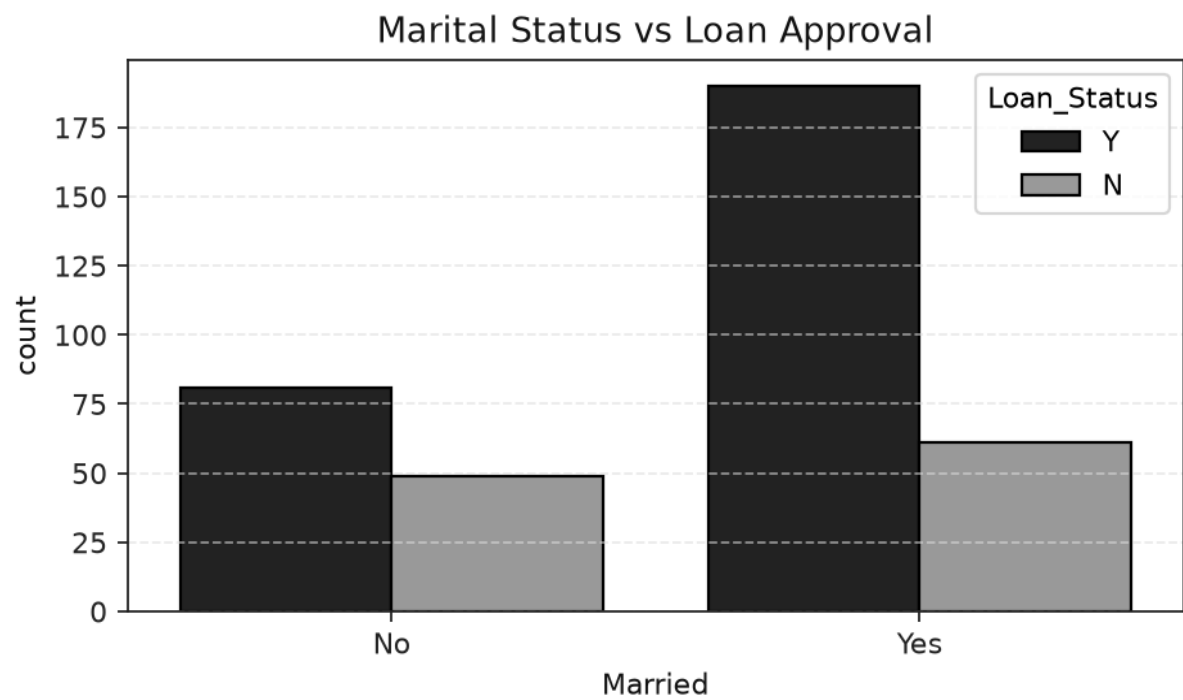


Fig 4 — Marital status vs Loan approval

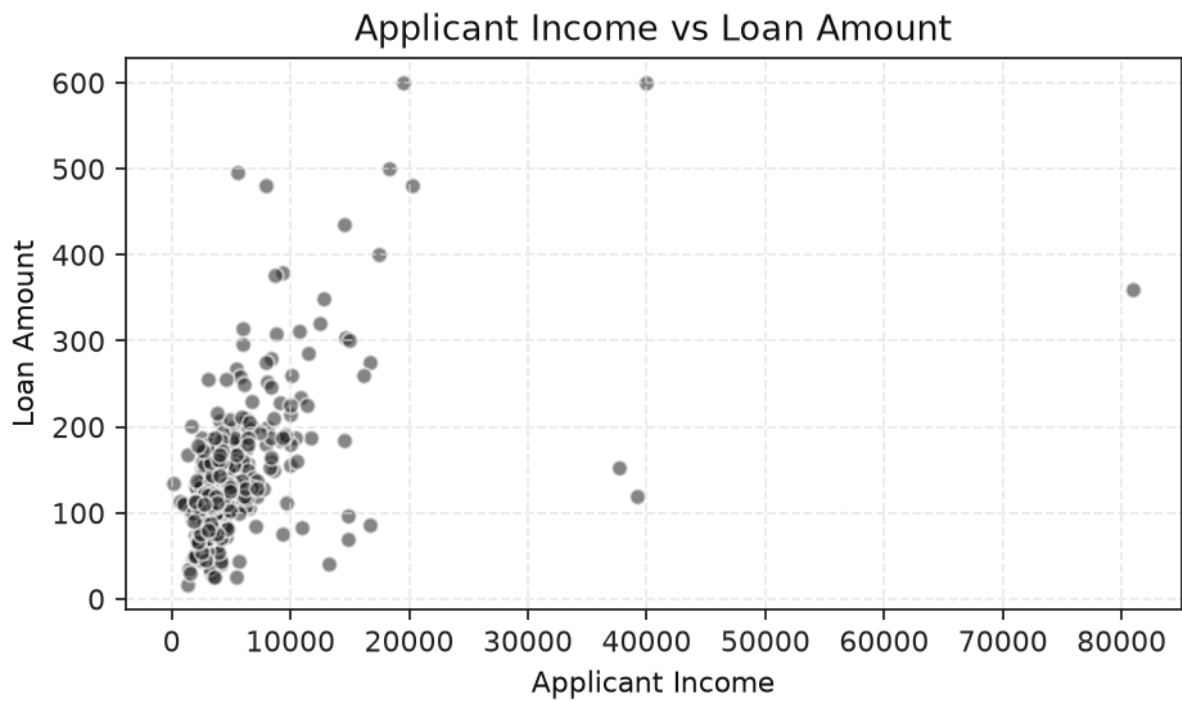


Fig 5 — Applicant Income vs Loan Amount

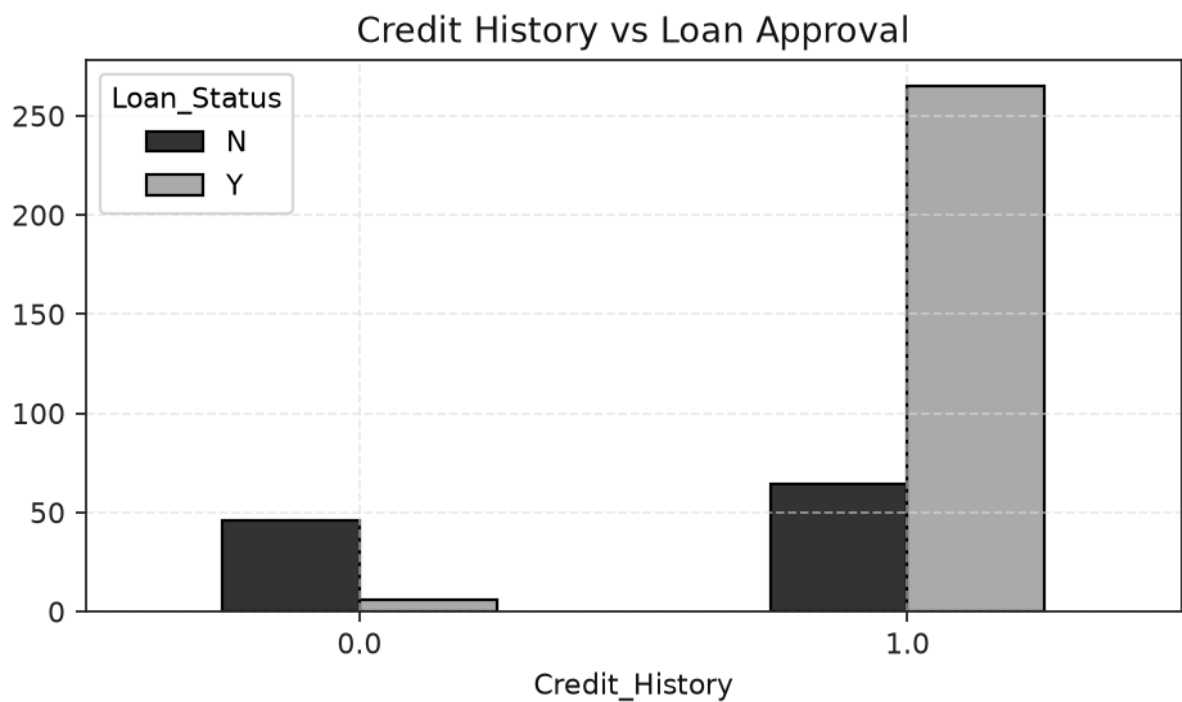


Fig 6 — Credit History vs Loan approval (the strongest predictor)

05 Feature Encoding & Correlation

Categorical features were converted to numeric form using **LabelEncoder**, and the **Loan_ID** column was dropped before computing correlations.

In [4]

```
from sklearn.preprocessing import LabelEncoder
cat = ['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'Property_Area', 'Loan_Status']
le = LabelEncoder()
for i in cat:
    data[i] = le.fit_transform(data[i])
data.drop('Loan_ID', axis=1, inplace=True)
```

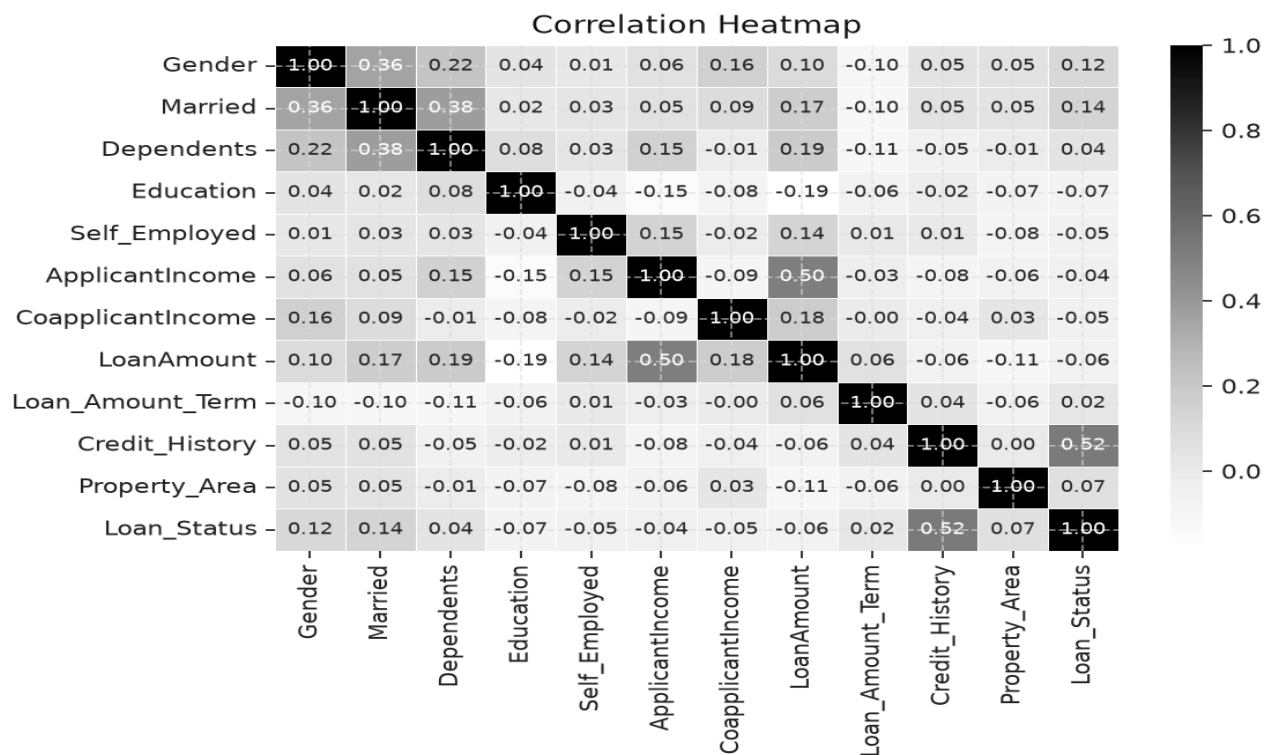


Fig 7 — Correlation heatmap of encoded features

06 Model Building

Train/Test Split, Scaling & SMOTE Balancing

In [5]

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from imblearn.over_sampling import SMOTE

x = data.drop('Loan_Status', axis=1)
y = data['Loan_Status']
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42, stratify=y)

sc = StandardScaler()
x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

sm = SMOTE(random_state=42)
x_train_sm, y_train_sm = sm.fit_resample(x_train, y_train)
```

Logistic Regression

In [6]

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression()
lr.fit(x_train_sm, y_train_sm)
pred_lr = lr.predict(x_test)
```

Out [6]

```
Precision : 0.7903
Recall    : 0.8909
F1-Score  : 0.8376
ROC AUC   : 0.7140
```

Random Forest

In [7]

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(x_train_sm, y_train_sm)
pred_rf = rf.predict(x_test)
```

Out [7]

```
Precision : 0.8226
Recall    : 0.9273
F1-Score  : 0.8718
ROC AUC   : 0.8021
```

07 Model Evaluation & Visualizations

Metric	Logistic Regression	Random Forest
Accuracy	0.7532	0.8052
Precision	0.7903	0.8226
Recall	0.8909	0.9273
F1	0.8376	0.8718
ROC_AUC	0.7140	0.8021

Tab 1 — Side-by-side metric comparison

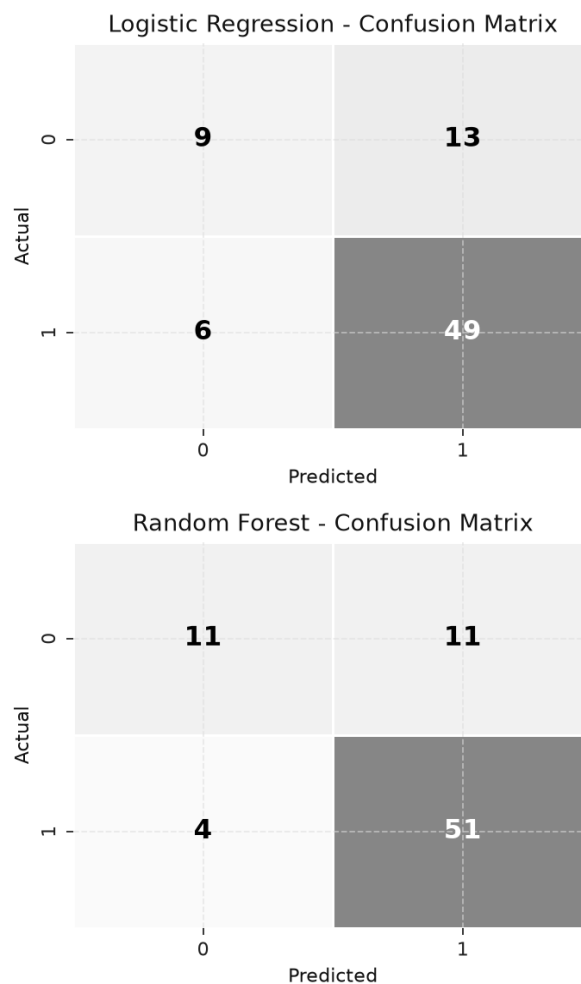


Fig 8 & 9 — Confusion matrices (Logistic Regression and Random Forest)

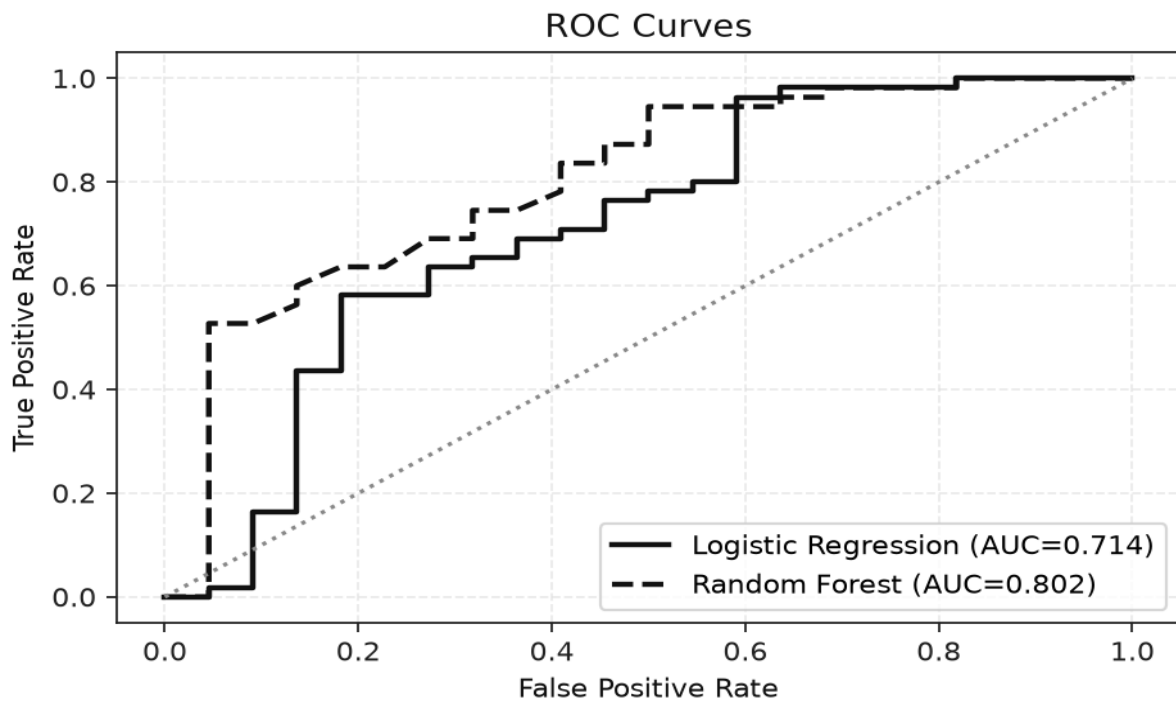


Fig 10 — ROC curves comparing the two classifiers

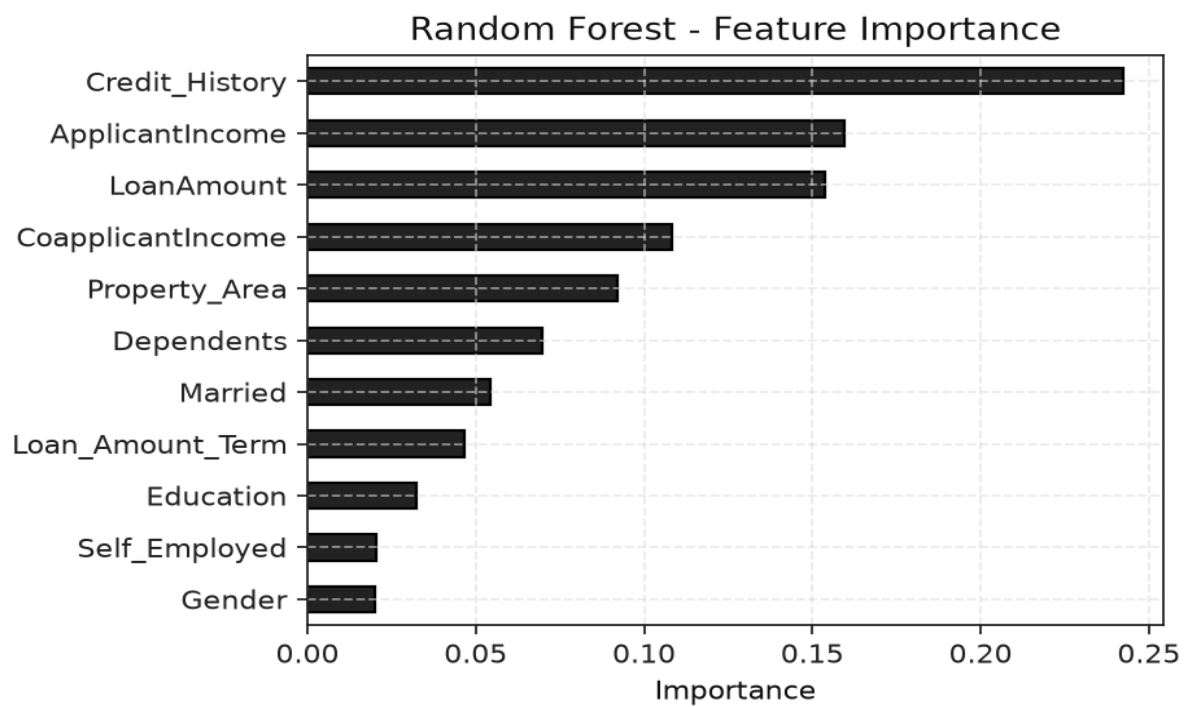


Fig 11 — Random Forest feature importance

08 Key Findings

#	Finding	Detail
1	Credit History dominates	Credit_History is by far the strongest signal — applicants with a clean credit record are
2	Income is necessary but not sufficient	ApplicantIncome and LoanAmount are correlated, but income alone does not guarante
3	Education has a moderate effect	Graduates show a higher approval rate than non-graduates, though the gap is smaller
4	Marital and self-employment status matter less	These features carry weak predictive power in both models.
5	SMOTE materially helps recall	Balancing the training set with SMOTE improves the model's ability to correctly flag re
6	Random Forest leads on AUC	RF achieves ROC-AUC = 0.802 vs LR = 0.714, indicating better ranking of approval p

09 Recommendations

#	Recommendation	Rationale
1	Prioritize credit-history verification	Make Credit_History the gatekeeper field — incomplete or unverified entries should tri
2	Use Random Forest as the production model	Random Forest provides the better ROC-AUC and handles non-linear interactions; ke
3	Monitor and re-train periodically	Income distributions and approval policies shift; schedule monthly retraining and drift m
4	Reduce bias risk	Audit predictions across Gender, Married and Property_Area to ensure no demograph
5	Improve data collection	Reduce missingness in LoanAmount and Credit_History at intake — these two column
6	Explore advanced models	Try Gradient Boosting / XGBoost and threshold tuning for further uplift in recall on reje

10 Notebook & References

Notebook File

- **task2.ipynb** — End-to-end notebook with data loading, EDA, encoding, SMOTE, Logistic Regression and Random Forest training, and evaluation.

Dataset Source

- Kaggle — Loan Prediction Dataset: <https://www.kaggle.com/datasets/ninzaami/loan-predication>

Libraries Used

pandas, numpy, matplotlib, seaborn, scikit-learn, imbalanced-learn (SMOTE)

— End of Report —